LiteGrip

操作手册

NEXFORM ROBOTICS

版本：main　　日期：2026 年 11 月 2 日

修订记录

版本	修订时间	修订记录

目录

1 LiteGrip Studio 1
1.1 相关仓库 1
1.2 1. 装依赖 1
1.3 2. 跑起来 2
1.3.1 命令行选项 2
1.4 3. 接真机 3
1.5 4. 动之前先标定 4
1.6 界面 4
1.6.1 一直在的那一条 4
1.6.2 控制——让夹爪动起来 5
1.6.3 状态——读数与故障 6
1.6.4 曲线——两张图 6
1.6.5 窗口其余部分 6
1.7 标定 7
1.7.1 上位机用的是哪份 7
1.7.2 闸门 7
1.7.3 自动标定——测行程 7
1.7.4 手动两点标定——用手记两个点 7
1.7.5 选文件，以及让它保住 8
1.7.6 「来源」那一行是什么意思 8
1.7.7 电机标零——写入编码器零点 9
1.8 出问题了 9
1.9 已知限制 10
1.10 文件与设置 11
1.11 开发 11
1.12 许可证 12

1 LiteGrip Studio

English | **简体中文**
LiteGrip Studio 是 LiteGrip 两指自适应夹爪的操作台。拖滑块让夹爪走到指定位置，按一下按钮全开或全闭，按设定的力上限夹住东西，同时看位置、力、温度和故障的实时读数。它同进程 `import` LiteGrip Python SDK 走 SocketCAN，没有跨语言桥。
没接夹爪时它跑内置的仿真夹爪，**每一个控件都照常工作**——仿真不是界面的占位，它跑的是和真机同一套运动状态机。所以没机器也能装上、把每个页面点一遍、把测试套件跑通。
窗口是四个页签——控制、状态、曲线、标定——界面是简体中文，本文里出现的按钮名都是屏幕上原样显示的字。

1.1 相关仓库

仓库	角色
litegrip-python	Python SDK——已 vendor 在 `src/litegrip`
litegrip-cpp	C++ SDK
litegrip-docs	产品文档
litegrip-ros2	ROS 2 驱动

1.2 1. 装依赖

需要 Python 3.10 或更高，和三个库，装进你 `python3` 指向的那个解释器：

```
python3 -m pip install PyQt5 pyqtgraph numpy
```

跑测试套件再加`python3 -m pip install pytest`,打包再加`python3 -m pip install pyinstaller`。SDK 不用装——已 vendor 在 `src/litegrip`。
有两件事不要做：

- **别用 `pip install -e .`**。`pyproject.toml` 里的版本号是 semantic-release 的占位符，不是合法的 PEP 440 版本号，setuptools 会拒绝构建。
- **别从 PyPI 装 `litegrip`**。上位机兜底用的那套标定数值，是从它 import 到的那个 `litegrip` 身边读的；装一个同名的发行包，会静默换成另一套数值。

1.3 2. 跑起来

```
./run_litegrip_studio.sh sim        # 上位机，对着内置仿真夹爪
./run_litegrip_studio.sh gui        # 上位机，对着 can0 上的真机
./run_litegrip_studio.sh selftest   # 自检：不需要 Qt、不需要硬件
./run_litegrip_studio.sh test       # 测试套件
./run_litegrip_studio.sh build      # 单文件可执行程序，输出到 dist/
```

先跑 `sim`：不需要硬件，跑的却是同一套代码。
`selftest` 只跑纯逻辑——单位换算、标定方向、零点不属于本机时拦下、被控对象收敛、运动到位、力矩上限、急停失能——**完全不 import Qt**，所以在 PyQt5 坏掉的机器上照样能跑。每项一行 PASS / FAIL，最后给一个通过数。
除了这个仓库，什么都不用装、不用额外检出：SDK 已 vendor 在 `src/litegrip`，启动脚本会把 `src/` 挂上 PYTHONPATH。
启动脚本用的是 python3。依赖装在虚拟环境里时，用 PYTHON_BIN 指过去：

```
python3 -m venv .venv
.venv/bin/python3 -m pip install PyQt5 pyqtgraph numpy
PYTHON_BIN=.venv/bin/python3 ./run_litegrip_studio.sh sim
```

报 `ModuleNotFoundError: No module named 'PyQt5'`，说明依赖装到别的解释器去了。 `./run_litegrip_studio.sh -h` 会打印启动脚本实际用的那个（**当前解释器：…**）；往那个里装，或者把 PYTHON_BIN 指到你装的那个。
多出来的参数原样透传给程序，例如 `./run_litegrip_studio.sh sim --log-level DEBUG`；`-h` 列出全部子命令。

1.3.1 命令行选项

选项	默认	作用
`--backend {sim, real}`	`real`	用仿真被控对象代替真机
`--can-channel NAME`	`can0`	用哪个 SocketCAN 接口
`--can-bitrate INT`	`1000000`	连接时把接口配成这个波特率
`--no-can-setup`	关	连接时不准备接口，也不弹授权框
`--can-id INT`	取自标定文件	电机 CAN ID
`--mst-id INT`	取自标定文件	上位机 CAN ID

选项	默认	作用
`--calibration PATH`	记住的路径，然后包内那份	要加载的标定文件
`--log-level LEVEL`	`INFO`	控制台日志级别；日志文件始终记 `DEBUG`
`--log-file PATH`	`~/.litegrip/logs/litegrip-studio.log`	日志写到哪
`--selftest`	关	只跑不需要 Qt、不需要硬件的自检，然后退出
`--version`		打印版本号后退出

1.4 3. 接真机

不用自己先把接口弄起来。按「连接」，上位机会先看 `can0`，只在接口确实不对时才动手：

- 提权走 `pkexec`，也就是桌面自己的授权框。**上位机全程看不到密码**，所以也不可能把密码写进日志。
- 接口本来就是配好的，就不弹框、也不跑任何提权命令——所以你自己手动把 `can0` 弄起来之后，不会每次连接都被问一遍。
- 配置里带 `restart-ms 100`：控制器进入 bus-off 后 100 ms 自己重开，而不是一直躺着。**不是每个适配器都支持这一项**。遇到一个拒绝它的，上位机会去掉这一项、照样配上、并在日志里说明；而在这类适配器上，bus-off 的控制器不会自己恢复，下次「连接」会重新配一次。
- CAN FD 接口只报告、不改：SDK 会自己识别 FD 并配合它工作，而把接口改回经典 CAN 会连带改掉那条总线上别的节点。
- 授权框被取消、`can0` 不存在、`ip` 命令失败：**连接尝试照样继续**，日志和横幅会说明发生了什么。

如果接口由某个 unit 文件或脚本管着、你不想要那个授权框，加 `--no-can-setup`；波特率不是 1 Mbit 就用 `--can-bitrate`。
要手动做，等价的三条命令是：

```
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 restart-ms 100 fd off
sudo ip link set can0 up
./run_litegrip_studio.sh gui
```

中间那条报 “Device doesn’t support restart from Bus Off.”，说明你的适配器不支持 `restart-ms` ——去掉这一项，跑另外两条。
如果前两条都成了、最后一条报 “RTNETLINK answers: No such file or directory”，那是适配器丢了 USB 端点表，任何 `ip link` 命令都救不回来。把适配器拔下来再插回去，或者跑 `sudo modprobe -r gs_usb &&`

`sudo modprobe gs_usb`——后者不用碰线就做同样的事，代价是把机器上 **所有** `gs_usb` 接口一起重置。按「连接」遇到这个时，上位机自己会做这件事。

1.5 4. 动之前先标定

上位机启动时用的是 **SDK 包里带的出厂标定文件**，它是在出厂的那一台夹爪上录的。同型号的另一台夹爪并不是另一台机器：读数还在同一个量级，行程两端也大致在预期位置上。两台机器之间真正差的是加工公差——夹爪、连杆、编码器零点——所以读数可能略有偏差，而标定去掉的正是这一点。
所以上真机之后，先到「标定」页跑**自动标定**，再去依赖屏幕上的数字。在有一份属于**你这台**夹爪的标定之前，屏幕上的数字当「接近」看，别当「精确」看。
标定页显示**运动闸门：已阻断**（`BLOCKED`）时，在你处理掉它之前什么都不会动。这是有意设计的：上位机宁可拒绝运动，也不拿它没法担保的数字去驱动。详见标定。

1.6 界面

四个页签，加上始终可见的连接栏、急停按钮，和底部日志坞。

页签	里面有什么
控制	位置滑块、动作按钮、速度与夹持力
状态	实时读数、温度、故障、链路健康，以及电机维护动作
曲线	位置曲线和力曲线
标定	标定：两个探针、文件操作、闸门

1.6.1 一直在的那一条

连接与电源里有六个按钮和两个状态灯：

按钮	作用
连接	连上夹爪（工作期间变成「连接中…」）
断开	断开
使能	电机使能——不使能什么都不会动
失能	电机失能

按钮	作用
清除故障	清除已闩锁的故障
急停复位	复位已闩锁的急停

每个按钮只在它的命令确实能被接受时才可点，所以「使能」是灰的说明上位机还没连上，不是按钮坏了。
急停 (Esc) 立刻零力矩并失能，而且**闩锁**：在你按「急停复位」之前，一切运动都被拒绝。它绑在 **Esc** 键上，而且在输入框聚焦时、在弹窗打开时都有效——这是唯一一个被有意允许打断一切的快捷键。

1.6.2 控制——让夹爪动起来

位置（拖动即控制，松手执行）就是那个滑块。拖动设定目标，**松手时**才下发命令。跟着它动的有两个标记：

- **青色细线**是夹爪要去的地方——最后一次命令；
- **绿色三角**是夹爪实际所在的位置，来自遥测。

旁边的读数显示**实际位置、目标、误差**。**夹住东西的时候只显示实际位置**：一次夹取是在力上限下往 0 mm 顶到底，所以它的目标永远是闭合端、它的误差就是物体宽度——两个都属实，但并排摆着看，像一次跑歪了的运动。

控件	作用
全部闭合	按一下走到闭合位置
全部张开	按一下走到最大张开
拖动时实时跟随	拖动过程中就下发目标，而不是松手才发
停止（保持位置）	停下来，保持当前角度
零重力（可手掰）	零刚度零力矩：电机保持使能，但可以用手掰动夹爪

速度是运行速度，5–150 mm/s，默认 50 mm/s。
夹持力是夹取时的力上限，3–40 N。超过 35 N 变橙、到 40 N 变红，这 40 N 是机构的额定值，不是一个让你照着顶的目标。下限是控制台允许的最小设定。夹取走 SDK 自带力矩预算的 `grasp()`，过去的 6 N 下限不再适用；现在是 3 N，因为本机上 2 N、2.5 N 都还是勉强夹住（2026-11-02），而且 SDK 在设定值低于 2.2 N 时会拒掉「接近段根本没动」的夹取。想填更小的值，输入框会拦在 3 N。它旁边有两个按钮：

按钮	作用
夹取 N N	合上夹爪夹住物体，并保持在设定的力上
放开	张开到离所夹物体 10 mm 处——从**夹住时那个位姿**量起，所以这 10 mm 是物体实际得到的让位

停止（保持位置）和**零重力（可手掰）**在闸门 BLOCKED 时照样可用。闭合、张开、夹取、放开会被拒绝，所以标定不可用的时候，能挪动夹爪的路子只有零重力和你的手。

1.6.3 状态——读数与故障

实时读数显示**位置、速度、夹持力、力矩、速度参考、误差、MOS 温度、线圈温度**。温度行在离上限 10 °C 时变橙、到上限变红（MOS 70 °C，线圈 80 °C）。
链路与回路显示**运动状态、已抓取、状态帧、帧速率、链路静默、tick 耗时、tick 超时、错误码**。
连接与故障给出一个链路灯和一个故障灯。故障闩锁时，下面的提示会说明查什么：欠压 → 查 24 V 供电；过流 → 查机械是否卡死；过温 → 等 MOS 或线圈冷却。
维护里只有一个动作：**电机标零…**。见标定。

1.6.4 曲线——两张图

两张图共用一条时间轴，画最近 10 秒：

- **位置 (mm)——实测**（实线）对**目标**（虚线）。纵轴按标定行程缩放。
- **夹持力 (N)——力**。纵轴按窗口里的数据自适应，所以一次 12 N 的夹取会占满整张图，而不是缩在 40 N 轴的下面三分之一里。

暂停冻住它们，**清空**倒掉它们；恢复后缓冲清空、从头画起，图下面的提示说了这件事。

1.6.5 窗口其余部分

- 页签上方是**告警横幅**，一次只挂一条。描述某个状态的告警——故障、急停、某次拒绝——会在状态不再成立时被自动撤下，所以一台恢复了的控制台不会一直看着像坏着。只报告单次事件的告警——写盘失败、一次探针结束——会留到被下一条顶掉。所有内容（包括成功）都在日志里。
- 底部是**日志**坞。**显示调试信息**把调试行也显示到界面上；日志文件无论如何都记着它们。
- 状态栏里的**深色 / 浅色**切主题。控制台默认深色。你没切之前什么都不写，所以「没人选过」和「选中的正好是默认」两种状态分得开。
- 状态栏还显示**位置**和**闸门**——闸门就是 READY 或 BLOCKED。

1.7 标定

1.7.1 上位机用的是哪份

有两级来源，由上位机自己判断用哪级：

1. **你指定的文件**——`--calibration`、设置文件里记住的路径，或 `$LITEGRIP_CALIB`。**这才是描述你这台夹爪的文件**，也是唯一能离开默认的路。
2. **SDK 包里带的标定文件**（`litegrip/factory_calibration.json`）——什么都没指定时，上位机跑的就是它。

本仓库把包内这一份**覆盖成了本机手推记录的那组值**，而不是上游用探针测出来的；这次分叉记在 `src/litegrip/VENDORED.md` 里，含它自己的哈希。
指定的文件不存在，或者存在但不可用，都会如实报出来并**到此为止**——上位机不会悄悄退回出厂数值，因为「拿一份你没选的文件的数字去运动、而界面上什么都看不出来」是所有可能里最差的那个。

1.7.2 闸门

标定页上的**运动闸门**，和状态栏里的**闸门**，只有两种值：`READY` 或 `BLOCKED`。
`BLOCKED` 表示上位机不会下发运动：完全没有标定、文件没通过校验、指定的文件不存在、标定属于另一台机器，或者探测结果还没写盘。页面会说明是哪一种。这道判定在下发任何帧之前做，不是在控件上做。
闸门关着的时候，**停止**和**零重力（可手掰）**仍然可用——文件才是问题所在时，零重力就是你把夹爪挪开的那条路。

1.7.3 自动标定——测行程

需要电机已使能。按**开始自动标定**，上位机会把夹爪顶向张开极限、再顶向闭合极限，每一步都让你确认：**已到张开极限，确认**和 **已到闭合极限，确认**。页面下方的进度条显示当前阶段（正在顶向张开极限、正在退回、正在顶向闭合极限）和实时的**实测位置**（毫米与弧度两个值）。**取消标定**可以停下。
探针必须顶到硬限位上——它找的就是硬限位——所以夹爪会往行程两端各压一下。它测的是两个硬限位之间的距离，而且它是这个程序里唯一被允许在没有标定时运动、以及走到标定行程之外的动作：它就是产生标定的那一步。

1.7.4 手动两点标定——用手记两个点

用在硬限位够不到、或者测不出来的时候——连杆太紧、驱动器不肯走探针步。**开始手动标定**会把轴切到零重力并明说这一点；退出时（包括取消）上位机会把轴交回来，因为整个过程你的手都在夹爪上。
然后：

1. 用手把夹爪推到**张开**极限，按**记录张开极限**。
2. 推到**闭合**极限，按**记录闭合极限**。这一点就是 0 mm。

你按的是哪个按钮，就是「哪一端是零点」的全部答案，所以顺序不对的按键会被明确拒绝，而不是被当成「轮到哪个点算哪个点」。每个点的等待都有上限，5 分钟：整个过程轴都是松的，而一台挂着「已使能但零刚度」电机的控制台，就是一个会砸在下面东西上的夹爪。

1.7.5 选文件，以及让它保住

按钮	作用
载入文件…	挑一个标定文件并真的读它。之后页面会显示这些数字来自哪个路径。
刷新显示	重读**当前生效的那份文件**，且只读它

你载入的那份，就是下次启动会加载的那份。载入不会因为重启而作废。所以记住的文件没了之后，出路就是「载入文件…」：挑另一份，或者挑 SDK 包内的 `litegrip/factory_calibration.json` 退回出厂标定——这时控制台**忘掉**记住的路径，而不是把包内这份写成「你选的」标定。当前生效的文件被删掉时，「刷新显示」也会告诉你同样的事，并指出该载入哪份出厂标定。

探针跑到终点时会**问你**——弹窗上是**应用标定**和**暂不**，并说明结果会写到哪个文件。你不回答就什么都不写，因为按「记录」是要求**测量**，不是要求按测出来的值去动。

- **应用标定**写盘，并让后端把刚写下的文件读回来；来源转成**用户标定**，闸门转 READY。此后每次启动都加载这个文件。
- **暂不**把结果留在内存里。闸门照旧关着，夹爪保持在探针测到的角度，不会弹回去。

写盘有两件事它拒绝做，并会说明原因：校验没通过的结果——它要覆盖的是一份能用的标定；以及 SDK 包里自带的那份——它描述的是录制它的那台夹爪。两种拒绝都会再问一次，所以「应用标定」自己就是重试。

1.7.6 「来源」那一行是什么意思

标签	含义
用户标定	属于你这台夹爪的标定文件
出厂标定	SDK 包里带的那份——默认值，不是你这一台的测量结果
标定无效	没通过校验的用户文件
未标定	完全没有文件
内存标定（未保存）	探测结果还在内存里、没写盘；写下来之前闸门一直关着

同一个框里的**显示技术详情**会展开上位机实际在用的那些原始数值。

1.7.7 电机标零——写入编码器零点

在**状态**页的**维护**区。**电机标零…** 告诉驱动器：夹爪此刻所在的位置就是角度零点。这个偏置写进电机自己的 flash，**没有撤销**——上位机读不回来，也放不回去。
它是维护动作，不是标定的一步：它**不重新测量任何东西**。夹爪和数字已经偏得远到「再测一遍两端」才是正事的时候，就是该用它的时候。
确认框在写入前列出三条后果，写入后横幅再重复一遍：

- **必须重新标定**。文件里的跨度长度不变，但它的两个角度、以及由它们算出的每一个毫米，现在一起被偏置了。在你用自动标定或手动探针重做之前，这些都一起是错的。
- **闸门拦不住它**。在限位上取的零点，把整个坐标系挪动的量远小于闸门校验的那个容差，所以运动不会被拦住。这就是为什么这里是用大白话把后果写清楚，而不是由上位机拒绝运动。
- **电机会短暂失能**。 SDK 会先失能驱动、写偏置、再使能，所以大约一秒之内夹爪可以用手推动，而且写入结束之前按 Esc 不生效。确认之前把手和工件都拿开。

按**写入零点**确认。这条警告只由**已了解**按钮撤掉，别的方式都不行——循环里没有任何东西能撤它，因为拿旧偏置测出来的标定不会自己变对。
标零需要上位机已连接、电机已使能；探针运行期间也会被拒绝。仿真后端直接拒绝，报「本后端不支持标零」。

1.8 出问题了

你看到的	是什么意思，怎么办
`ModuleNotFoundError: No module named 'PyQt5'`	依赖装到别的解释器去了。`run_litegrip_studio.sh -h` 会打印实际用的那个——往那个里装，或设 `PYTHON_BIN`
连接之后报 “Network is down, errno 100”	接口没起来或者处在 bus-off。这是链路问题不是电机问题：检查适配器和接线，然后重连
报 “RTNETLINK answers: No such file or directory”	USB-CAN 适配器丢了端点表。拔下来重插，或者 `sudo modprobe -r gs_usb && sudo modprobe gs_usb`
按「连接」没反应，也没弹授权框	授权框被取消了，或者 `can0` 由某个 unit 文件管着。用上面那三条命令手动把接口弄起来
运动闸门：已阻断	上位机不会动。页面会说是哪一种原因：没有文件、文件无效、文件不存在，或者零点属于另一台机器。「零重力（可手掰）」照样可用

你看到的	是什么意思，怎么办
标定文件已不存在：…	记住的那份文件被删了。「载入文件…」可以挑另一份，或者挑 SDK 包内的 `litegrip/factory_calibration.json` 退回出厂标定
夹爪完全不动	先看闸门，再看电机是否已「使能」，再看急停横幅：闩锁的急停会拒绝一切运动，直到按「急停复位」
运动被拒绝，说「没有读到位置」	电机还没上报过任何状态帧。先使能，等**状态帧**开始计数
电机标零报「本后端不支持标零」	你在仿真后端上，它是有意拒绝标零的
力曲线看着像噪声	空载时力矩只是一点几分之一牛顿。夹个东西，或者无视它

1.9 已知限制

- **仿真后端拒绝电机标零**。这是有意为之，不是漏做：仿真里那个位置同时是机构的物理状态和它报出的角度，只挪读数的话，它就会按一个和自己说法对不上的状态积分。
- **SDK 自带的 `calibrate()` 不提供**。它不可中断且会跑约 24 秒运动——与其提供一个急停无效的 24 秒窗口，不如只上标定页上那两个探针。
- **出厂标定是默认值，不是你这台夹爪的测量结果**。它是在出厂那一台夹爪上录的，所以用在同型号的另一台夹爪上，读数是接近而不是精确，差的就是那一台的加工公差。上位机不去判断这件事——但它拦得住另一种：文件的编码器零点属于另一套坐标系（拆装过的那一台，或者另一个型号），因为这种文件的行程不包含夹爪实际报出的那个角度，闸门会在下发任何帧之前转 `BLOCKED`。
- **行程是本台机器的常量**，`constants.DEFAULT_TRAVEL_MM`，85.0 mm。它同时决定毫米换算系数和滑块量程，所以被改成别的数的控制台会把每个毫米都报错、而界面上看不出异常。把这台控制台挪到另一台夹爪上，意味着改这个常量并重新探测——没有任何命令行参数、设置项或控件能改它。
- **力上限是机械额定 40 N**，在 `constants.FORCE_MAX_N`。理论的 100 N 只作灰色不可选的注释展示。
- **能设定的最小力是 3 N**，在 `constants.FORCE_MIN_N`。夹取走的是 SDK 自带力矩预算的 `grasp()`，本机上 2 N、2.5 N 都还是勉强夹住（2026-11-02），所以下限是 3 N；它同时要高于 SDK 判拒夹的 2.2 N。下限只负责让 0 仍然是「不限力」这个模式，而不是一次很轻的夹取。模拟器仍然跑控制台自己的 FSM，它的接触检测大约需要 6 N，所以模拟环境里贴近下限的夹取可能很晚才判定接触；实机不受这个限制。
- **量程顶端停在记录到的张开极限内侧 1 mm**。探测是在 40 N 压力下把张开极限压出来的，连杆在这股力下让出约 1 mm，所以直接命令到记录极限，等于每次全开都把止挡压一下。闭合端不做内缩：0 就是标定出来的零点。

1.10 文件与设置

上位机把一个用户的所有东西都放在 `~/.litegrip/` 下：

路径	是什么
`litegrip_studio.ini`	设置：上次的速度和力、滑块是否拖动即跟随、记住的标定路径、主题、窗口几何、日志调试开关
`litegrip_calibration.json`	标定默认的保存位置——但**放在这里并不会让它生效**。上位机读的是它记住的那个路径
`logs/litegrip-studio.log`	日志，512 KiB 轮转、保留三份，始终按 DEBUG 记录

环境变量：

变量	作用
`PYTHON_BIN`	启动脚本和构建脚本用的解释器
`LITEGRIP_SDK_PATH`	用一份 SDK 检出代替 vendor 的那份；给检出根目录或它的 `src/`
`LITEGRIP_CALIB`	顶掉默认的用户标定路径
`LITEGRIP_FACTORY_CALIB`	用你指定的一个路径顶掉出厂标定文件

1.11 开发

测试不需要硬件，也不需要安装 SDK：

```
./run_litegrip_studio.sh test
# 等价于
QT_QPA_PLATFORM=offscreen python3 -m pytest tests/ -q
```

`QT_QPA_PLATFORM=offscreen` 是没有显示器时也能让 UI 测试建出真实控件的原因。`tests/conftest.py` 把 `HOME` 指到本次会话自己的临时目录，所以任何测试都读不到、也写不到你的 `~/.litegrip/`；仓库根

目录的 `conftest.py` 还把 `src/` 插进 `sys.path`，vendor 进来的 SDK 只差这一步就能 import。
SDK 是 **vendor 进来的，不是依赖**：`src/litegrip` 里存着 litegrip-python 的一个固定 commit，所以 clone 下来什么都不用装就能跑。**不要改它下面的任何文件**——就地改动只能活到下一次 vendor，那时每个文件都是整份覆盖的。确实不得不有分歧时，在 `src/litegrip/VENDORED.md` 里按文件记下来；那份文件同时记着整棵树的哈希和重新 vendor 的步骤，`tests/test_vendored_sdk.py` 会重算那个哈希，所以就地改一处就会让测试变红。今天没有任何本地覆盖：那里的每个文件都是上游的。
`./run_litegrip_studio.sh build` 在 `dist/` 下产出单文件可执行程序。发版由 semantic-release 在每次合并进 `main` 时自动完成；git tag 是版本号的唯一事实来源，`pyproject.toml` 里的 `0.0.0-semantic-release` 是占位符，不要手改。
上面这些「为什么」——架构为什么长这样、滑块背后到底在做什么、标定闸门为什么放在那个位置——都在 `DESIGN.md`（英文）。本文是使用手册。
本仓库遵循 NEXFORM ROBOTICS 的共享仓库规范：`.nexform-standards.json` 里的智能体作业规则、Conventional Commits，以及自动化的 semantic-release 版本号。

1.12 许可证

以 **Apache License 2.0** 授权 —— 见 `LICENSE`。
Copyright © 2026 NEXFORM ROBOTICS.